Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Programa de Licenciatura en Ingeniería en Computadores

Curso: CE-4301 Arquitectura de Computadores I



Especificación Proyecto Grupal I

Profesor:

Dr.-Ing. Jeferson González Gómez

Fecha de entrega: Miércoles 14 de Octubre, 2026

Proyecto Grupal I – Arquitectura VLIW Propia para Aplicaciones de Cifrado por Bloques

Grupo: 5 estudiantes

1. Objetivo

Mediante el desarrollo de este proyecto, el estudiante aplicará los conceptos de arquitectura de computadores en el diseño e implementación en SystemVerilog de un procesador **VLIW** (*Very Long Instruction Word*) con una arquitectura del set de instrucciones (ISA) propia, orientada a la aceleración de un algoritmo de cifrado por bloques tipo Feistel. El proyecto se evaluará explícitamente en **dos partes**: (1) el diseño de la Arquitectura del Set de Instrucciones (ISA), incluyendo el formato de bundle/slots y los tipos de unidad funcional que la componen; y (2) su organización interna (incluyendo el pipeline) y la implementación en SystemVerilog, evaluada a nivel de **simulación** utilizando Icarus Verilog (iverilog) `https://steveicarus.github.io/iverilog/` o Verilator `https://www.veripool.org/verilator/`. Adicionalmente, los estudiantes diseñarán una herramienta que permita cargar archivos de cualquier formato (texto, imágenes, binarios, etc.) en la memoria de la simulación para cifrar y descifrar datos reales.

2. Descripción General

Las arquitecturas VLIW explotan el paralelismo a nivel de instrucción (*Instruction-Level Parallelism*, ILP) de forma **estática**: en lugar de que el hardware detecte y reordene instrucciones independientes en tiempo de ejecución (como en un procesador superescalar), es el **compilador** (o el programador) quien agrupa explícitamente varias operaciones independientes en una sola palabra de instrucción larga (*bundle*), cada una destinada a una unidad funcional distinta que se ejecuta en paralelo durante el mismo ciclo. Esta filosofía de diseño, empleada históricamente en arquitecturas como Multiflow Trace, Intel/HP Itanium (IA-64) y los procesadores TI TMS320C6x para DSP, permite hardware más simple y eficiente en área/potencia, a costa de trasladar la complejidad de la calendarización (*scheduling*) al software.

En este proyecto se dará una introducción a este paradigma mediante el diseño de un procesador VLIW propio con soporte nativo para la aceleración de un algoritmo de cifrado de bloque simplificado tipo red de Feistel, al que llamaremos **Feistel4**. Es fundamental resaltar que, si bien el procesador incluye una unidad funcional dedicada a esta aplicación criptográfica, **la arquitectura y la organización deben soportar programas de propósito general ("tradicionales")**: todo programa VLIW válido debe poder incluir instrucciones aritméticas, lógicas, de acceso a memoria y de control de flujo, de forma completamente independiente de la aplicación de cifrado. La unidad criptográfica es una unidad funcional más dentro del procesador, no un reemplazo de las capacidades generales de cómputo.

Este proyecto además se articula con el curso de **Compiladores e Intérpretes (CE1108)**: cada grupo de CE4301 tendrá asignado un grupo de contraparte en CE1108, responsable de desarrollar un compilador que traduzca un lenguaje de alto nivel hacia el ensamblador de la arquitectura VLIW propia. La Sec. 6 detalla los requisitos y el contrato de interfaz entre ambos equipos.

3. Entregas Oficiales

El proyecto se divide en **dos entregas oficiales**, correspondientes a las dos partes evaluadas descritas en la Sec. 1: la especificación de la arquitectura, y la implementación funcional completa.

- **Entrega 1 – Especificación del ISA: viernes 25 de septiembre, 2026**, 11:59 p.m., vía TEC Digital. Debe incluir el documento de especificación del ISA: formato de bundle y slots, tipos de unidad funcional (Sec. 4.1, específicamente el formato de bundle/slots y las unidades funcionales – no incluye el pipeline, ver nota más abajo), el set de instrucciones, la *instruction reference sheet* con su codificación binaria, y la justificación de las decisiones de diseño. Esta entrega debe considerarse **congelada**: cualquier cambio posterior al ISA debe justificarse y comunicarse formalmente al grupo de contraparte de CE1108 (Sec. 6), ya que es este contenido – no la organización interna del pipeline – el que constituye el contrato con dicho grupo.
- **Entrega 2 – Implementación Funcional y Defensa Final: miércoles 14 de octubre, 2026**. Corresponde a la defensa presencial del proyecto completo: implementación en SystemVerilog, validación de la aplicación criptográfica Feistel4, ensamblador propio, herramienta de carga de archivos, e integración con el compilador de la contraparte de CE1108, según el detalle de la Sec. 8.

4. Especificación

4.1. Arquitectura VLIW

4.1.1. Formato de Bundle y Slots

Cada instrucción VLIW (*bundle*) debe estar compuesta por **al menos 4 slots**, cada uno dedicado a una unidad funcional específica y ejecutado en paralelo durante el mismo ciclo de reloj. Como mínimo, deberán existir **al menos tres tipos diferentes de unidades funcionales** entre dichos slots (por ejemplo: unidad aritmético-lógica, unidad de acceso a memoria, unidad de control de flujo y unidad criptográfica). Cada grupo deberá definir y justificar:

- El **ancho del bundle** (por ejemplo, 4 slots de 32 bits cada uno, para un bundle de 128 bits), y el formato de codificación de cada slot.
- La **asignación de slots a unidades funcionales**: puede optarse por un esquema de **slots fijos** (cada slot atado permanentemente a un tipo de unidad funcional, más simple de implementar) o un esquema **flexible** (más de un tipo de unidad funcional puede ocupar un mismo slot mediante un campo de selección), debidamente justificado en el documento de diseño.
- La **codificación de NOP** por slot, para los casos en que un ciclo no requiera ejecutar una operación en una unidad funcional particular.

Nota sobre calendarización estática: Al ser una arquitectura VLIW, el hardware **no debe** implementar detección ni resolución dinámica de riesgos de datos entre slots de un mismo bundle

o entre bundles consecutivos (i.e., sin *scoreboarding*, *forwarding* automático entre bundles, ni *stalls* por dependencias de datos). La responsabilidad de organizar las instrucciones en bundles válidos, respetando las dependencias de datos y de control, recae en quien genera el código (el ensamblador propio del grupo o el compilador de la contraparte de CE1108), tal como se detalla en la Sec. 6.

4.1.2. Unidades Funcionales

Como mínimo, el procesador debe incluir las siguientes unidades funcionales, cada una accesible desde su(s) slot(s) correspondiente(s):

- **Unidad Aritmético-Lógica (ALU):** suma, resta, AND, OR, XOR, desplazamientos, comparaciones.
- **Unidad de Acceso a Memoria (LSU):** instrucciones *load* y *store*.
- **Unidad de Control de Flujo (BRU):** saltos condicionales e incondicionales. Dado que el hardware no reordena ni detecta riesgos de control automáticamente, cada grupo deberá definir y justificar su estrategia frente a saltos (por ejemplo, *branch delay slot(s)* explícitos que deban ser llenados por el generador de código, o vaciado de *pipeline* sin penalización oculta).
- **Unidad Criptográfica (Feistel4):** instrucción(es) dedicadas para acelerar el algoritmo de cifrado descrito en la Sec. 4.2, con acceso exclusivo a la bóveda de llaves (Sec. 4.3) como fuente de subllaves.

Se permite (y se valorará positivamente, de forma justificada) incluir más de una unidad funcional del mismo tipo (por ejemplo, dos ALUs en dos slots distintos) siempre que se respete el mínimo de 4 slots y de 3 tipos diferentes de unidad funcional.

4.1.3. Organización Segmentada (Pipeline)

Nota: a diferencia del formato de bundle/slots y de los tipos de unidad funcional (que forman parte del contrato ISA de la **Entrega 1**, por ser lo que necesita el grupo de contraparte de CE1108 para generar código), el pipeline es una decisión de **microarquitectura/organización** que se diseña y evalúa junto con la implementación en SystemVerilog de la **Entrega 2** (Sec. 8); no es necesario congelarla ni entregarla en la Entrega 1.
La organización del procesador debe implementarse como un **pipeline** (segmentado): distintos bundles deben poder encontrarse simultáneamente en distintas etapas de ejecución, en lugar de que cada bundle se procese por completo (fetch a writeback) antes de iniciar el siguiente. Como mínimo, deberán distinguirse las siguientes etapas:

- **Fetch (IF):** búsqueda del bundle completo (todos sus slots) desde la memoria de instrucciones.
- **Decode/Dispatch (ID):** decodificación de los slots del bundle y despacho hacia sus unidades funcionales correspondientes.
- **Execute (EX):** ejecución en paralelo de las operaciones de cada slot en su unidad funcional (ALU, LSU, BRU, unidad criptográfica).

- **Writeback (WB):** escritura de resultados en el banco de registros y, cuando corresponda, en la bóveda de llaves.

Cada grupo puede proponer un número mayor de etapas (por ejemplo, separando el acceso a memoria de la escritura de resultados), debidamente justificado en el documento de diseño.
Nota: Dado que, como se indicó en la nota sobre calendarización estática (Sec. 4.1), el hardware no debe resolver riesgos de datos ni de control de forma dinámica, el pipeline **no requiere** *forwarding* ni *scoreboarding*: la responsabilidad de evitar dichos riesgos (mediante calendarización estática e inserción de NOPs) recae en quien genera el código. El pipeline sí puede requerir *stalls* por riesgos estructurales (por ejemplo, contención de un mismo recurso entre etapas), los cuales deberán documentarse y justificarse.

4.2. Aplicación de Cifrado por Bloques: Feistel4

Las redes de Feistel [1] son una construcción clásica para el diseño de cifradores de bloque simétricos: el bloque de entrada se divide en dos mitades (L, R), y en cada ronda se aplica una función de ronda F (parametrizada por una subllave) sobre una de las mitades, combinándola mediante XOR con la otra, seguido de un intercambio. Su principal ventaja arquitectónica es que la misma unidad de hardware puede reutilizarse tanto para cifrar como para descifrar, simplemente invirtiendo el orden en que se aplican las subllaves.
Para este proyecto se deberá utilizar una red de Feistel simplificada de **4 rondas**, llamada **Feistel4**, que opera sobre bloques de **64 bits** (dos mitades L, R de 32 bits cada una) utilizando una llave de **128 bits**, dividida en **4 subllaves de 32 bits** (una por ronda). En la Fig. 1 se presenta la implementación de referencia.

```c
#define ROL32(x, r) (((x) << (r)) | ((x) >> (32 - (r))))

// Funcion de ronda F: rotacion + suma modular 2^32, combinada con XOR
uint32_t feistel4_F(uint32_t x, uint32_t k) {
    uint32_t t = ROL32(x, 5) + k;
    return t ^ ROL32(x, 13);
}

// Cifrado de un bloque de 64 bits (L, R) con red Feistel de 4 rondas
void feistel4_encrypt(uint32_t *L, uint32_t *R, const uint32_t key[4])
    {
    for (int i = 0; i < 4; ++i) {           // 4 rondas
        uint32_t temp = *R;
        *R = *L ^ feistel4_F(*R, key[i]);
        *L = temp;
    }
}

// Descifrado: se recorren las mismas 4 subllaves en orden inverso
void feistel4_decrypt(uint32_t *L, uint32_t *R, const uint32_t key[4])
    {
    for (int i = 3; i >= 0; --i) {          // orden inverso de
       subllaves
        uint32_t temp = *L;
        *L = *R ^ feistel4_F(*L, key[i]);
        *R = temp;
    }
}
```

Figura 1: Implementación de referencia del cifrado/descifrado Feistel4.

4.3. Almacenamiento Seguro de Llaves (Bóveda)

Como se mencionó anteriormente, el algoritmo Feistel4 requiere una llave de 128 bits que debe protegerse adecuadamente: la eficacia y fortaleza del cifrado dependen en gran medida de que dicha llave se mantenga secreta y bien administrada.

Uno de los mecanismos empleados para evitar que las llaves sean atacadas (por ejemplo, si un atacante logra tener acceso al sistema) es no mantenerlas en archivos o memoria general, sino **almacenarlas directamente en el hardware** utilizando una *raíz de confianza* (*Root of Trust* – RoT), inaccesible para un usuario común del sistema, pero accesible para las aplicaciones seguras del sistema (e.g., la unidad criptográfica Feistel4).

Para este proyecto se deberá modelar una raíz de confianza en forma de una **bóveda de llaves** (memoria segura) que permita almacenar **al menos 4 llaves de 128 bits cada una** (cada llave de 128 bits corresponde a las 4 subllaves de 32 bits de una instancia de Feistel4, ver Sec. 4.2). Dicha bóveda deberá ser accesible directa y solamente por el CPU, por medio de instrucciones del set específicas para escribir u operar con los datos de la misma. La bóveda NO debe poder ser accedida como una memoria tradicional.

4.4. Requisitos Específicos de la Unidad Criptográfica para la ISA

4.4.1. Bóveda de Llaves

- Para la bóveda de llaves se deberá contar con una o más instrucciones específicas para almacenar llaves de 128 bits. Ya que sólo se permite almacenar 4 llaves, se deberá considerar esta limitante en el diseño de la(s) instrucción(es).
- Toda operación de cifrado/descifrado Feistel4 que involucre llaves deberá utilizarlas desde la bóveda: las subllaves consumidas por la(s) instrucción(es) de ronda Feistel4 (Sec. 4.4.2) deben provenir de una llave previamente almacenada en la bóveda, nunca de registros de propósito general cargados directamente por el programa.
- Los datos almacenados en la bóveda NO podrán ser escritos en registros del CPU o memoria general del sistema.
- Solamente la(s) instrucción(es) de ronda Feistel4 pueden acceder a la bóveda como operando fuente.
- **Control de acceso:** Se deberá implementar una o varias instrucciones específicas de autenticación/autorización (p.ej., *login*, *setpassword*, *authorize*, etc.) que aseguren que solamente código autorizado pueda acceder a la bóveda de llaves y realizar operaciones criptográficas. El procesador deberá mantener un registro de estado de autenticación que controle el acceso a estas operaciones privilegiadas. Las operaciones no autorizadas deberán generar una excepción o error de acceso.

4.4.2. Instrucción(es) de Ronda Feistel4

- El cómputo de Feistel4 debe acelerarse por hardware. Deberá incluirse en el set **al menos una instrucción** que ejecute una ronda completa de Feistel4 (i.e., que reciba L, R, y el índice de la llave/ronda a utilizar de la bóveda, y produzca el nuevo par (L, R) de esa ronda), de forma que un programa completo de cifrado/descifrado se construya encadenando 4 invocaciones de dicha instrucción con la llave correspondiente.
- Deberá considerarse un balance entre eficiencia y costo (área, complejidad) al diseñar la(s) instrucción(es) de la unidad criptográfica: una única instrucción que ejecute las 4 rondas completas resultaría en una unidad funcional excesivamente compleja y poco representativa del paradigma VLIW (donde el paralelismo se explota entre *slots*, no colapsando el algoritmo en una sola operación monolítica).
- La codificación exacta con la que la(s) instrucción(es) de ronda seleccionan la llave (0–3) y el número de ronda (0–3) dentro de la bóveda queda a criterio de cada grupo, debidamente justificada.
- Aunque la aplicación de referencia es el cifrado de archivos genéricos (Sec. 5.4), se permite y se valorará positivamente extender la validación a un caso de uso sobre **imágenes** (por ejemplo, cifrar/descifrar los píxeles de una imagen en un formato simple como PPM/BMP sin compresión, y visualizar el resultado), siempre que se mantenga como mínimo la validación sobre archivos genéricos.

4.5. Requisitos Generales de la Arquitectura del Set de Instrucciones

- La ISA debe estar organizada como un procesador **VLIW** de acuerdo con lo descrito en la Sec. 4.1 (bundle de al menos 4 slots, al menos 3 tipos de unidad funcional).
- Se deberá definir un mínimo de **8 registros de propósito general** de 32 bits cada uno. Se deja libertad a cada grupo para definir un banco de registros más amplio si lo considera conveniente (por ejemplo, para dar mayor margen de maniobra a la calendarización estática de instrucciones), debidamente justificado.
- Se debe incluir un **Program Counter (PC)** y un **registro de estado**.
- Todo programa VLIW válido debe poder combinar libremente, en cualquier bundle, instrucciones de los siguientes tipos (no exclusivos de la aplicación de cifrado):
  - Instrucciones Aritméticas y Lógicas (ALU): suma, resta, AND, OR, XOR, desplazamientos, comparaciones.
  - Instrucciones de Acceso a Memoria: load, store.
  - Instrucciones de Control de Flujo: branch condicionales, jump.
  - Instrucción(es) de la Unidad Criptográfica Feistel4 y de manejo de la bóveda de llaves (Sec. 4.4).
- El direccionamiento debe ser de **32 bits**.
- La memoria debe tener un tamaño mínimo de **64 KB**.

5. Implementación en SystemVerilog

5.1. Requisitos de Diseño

El procesador VLIW y toda la arquitectura deberán implementarse en **SystemVerilog**, siguiendo las mejores prácticas de diseño digital:

- **Modularidad:** El diseño debe estar dividido en módulos bien definidos (p.ej., `fetch`, `decode/dispatch` del bundle hacia los slots, `alu`, `lsu`, `bru`, `crypto_unit`, `key_vault`, banco de registros, memoria).
- **Parámetros:** Utilizar parámetros de SystemVerilog para definir constantes (número de slots, ancho de bundle, tamaño de memoria, número de registros, etc.) para facilitar mantenibilidad y experimentación.
- **Despacho paralelo:** La etapa de `decode/dispatch` debe decodificar y despachar, en el mismo ciclo, las operaciones de los distintos slots del bundle hacia sus unidades funcionales correspondientes.
- **Organización segmentada:** El procesador debe implementarse como un **pipeline** (Sec. 4.1.3), con registros de segmentación (*pipeline registers*) entre etapas que permitan que distintos bundles avancen simultáneamente por el datapath.

- **Testbenches:** Se deben crear testbenches exhaustivos para validar cada unidad funcional por separado y el sistema completo.
- **Documentación:** El código debe incluir comentarios explicativos en las secciones críticas.

5.2. Simulación con Icarus Verilog o Verilator

- El procesador será evaluado mediante **simulación**, sin necesidad de síntesis en FPGA. Cada grupo deberá elegir Icarus Verilog o Verilator como herramienta principal de simulación; se valorará positivamente (aunque no es obligatorio) que el Makefile soporte ambas.
- Se deberá proporcionar un archivo Makefile que compile y ejecute las simulaciones de forma reproducible (`make sim` o equivalente).
- Los testbenches deberán generar archivos de salida (.vcd o similar) para visualización en GTKWave.
- Se deberán incluir casos de prueba para:
  - Operaciones básicas de ALU, ejecutadas en paralelo con otras unidades dentro del mismo bundle.
  - Acceso a memoria (load/store).
  - Saltos condicionales e incondicionales.
  - Ejecución de la(s) instrucción(es) de ronda Feistel4.
  - Cifrado/descifrado completo (4 rondas) verificado contra la implementación de referencia (Fig. 1).
  - Manejo de la bóveda de llaves: almacenamiento de las 4 llaves, control de acceso (autenticación exitosa y fallida), y verificación de que las llaves no sean legibles desde registros de propósito general ni memoria general.
  - Un programa VLIW que combine, dentro de los mismos bundles, instrucciones "tradicionales" (ALU, memoria, control de flujo) junto con instrucciones criptográficas, demostrando que ambos tipos de aplicación coexisten en la arquitectura.

5.3. Ensamblador Propio

Debe existir un **ensamblador propio** (en el lenguaje de elección de quien lo desarrolle) que traduzca programas escritos en el lenguaje ensamblador de la arquitectura VLIW (incluyendo la organización explícita de instrucciones en bundles y slots) hacia el formato binario/hexadecimal utilizado para inicializar la memoria de instrucciones en la simulación (p.ej., mediante `$readmemh`). Su desarrollo puede ser asumido por el grupo de contraparte de CE1108 como parte de su propia herramienta de generación de código; sin embargo, de no contar con dicha herramienta en un momento dado, **el grupo de CE4301 debe ser capaz de producirlo por su cuenta**. En cualquier caso, este ensamblador es un requerimiento **independiente** del compilador que desarrollará el grupo de contraparte de CE1108 (ver Sec. 6) y debe permitir validar el 100 % del set de instrucciones diseñado sin depender de dicho compilador.

5.4. Herramienta de Carga de Archivos

5.4.1. Descripción

Se deberá desarrollar una herramienta (en C, Python o similar) que permita cargar archivos de **cualquier formato** (texto, imágenes, binarios, etc.) hacia la memoria del simulador. Esta herramienta facilitará el testeo de las operaciones de cifrado sobre datos reales, permitiendo cargar archivos de diferentes tipos y tamaños para operar sobre ellos con las instrucciones Feistel4 implementadas en la ISA.

5.4.2. Requisitos

- La herramienta deberá leer un archivo de entrada y convertirlo en un formato que pueda ser inyectado en la memoria de la simulación.
- Se debe generar una inicialización de memoria en formato **Verilog** (archivo .mem o similar) que contenga los bytes del archivo.
- La herramienta deberá permitir especificar la dirección de memoria inicial donde se cargará el archivo.
- Se deberá proporcionar información sobre el tamaño del archivo cargado y la zona de memoria utilizada.
- Se deberá proporcionar una utilidad para extraer datos cifrados/descifrados de la memoria de la simulación y guardarlos en un archivo de salida, permitiendo verificar el correcto funcionamiento con archivos reales (incluyendo, opcionalmente, imágenes).

5.4.3. Interfaz

La herramienta deberá tener una interfaz simple tipo línea de comandos:

```
./load_file.py --input archivo.txt --output memory.mem --address 0
   x1000
./load_file.py --input imagen.bmp --output memory.mem --address 0x2000
```

Para extraer datos:

```
./extract_data.py --memory memory_dump.txt --address 0x2000 --size
   4096 --output resultado.bin
```

6. Integración con el Curso de Compiladores (CE1108)

Este proyecto se desarrolla en conjunto con el curso de **Compiladores e Intérpretes (CE1108)**. Cada grupo de CE4301 tendrá asignado un **grupo de contraparte** en CE1108, responsable de diseñar e implementar un compilador (o herramienta equivalente de generación de código) capaz de traducir un lenguaje de alto nivel, definido por el grupo de CE1108 en coordinación con su profesor, hacia **código binario** (los bundles VLIW codificados en el formato de la arquitectura propia) ejecutable directamente sobre el procesador diseñado por el grupo de CE4301. El compilador debe llegar hasta binario: ya sea generándolo directamente, o generando primero ensamblador propio y traduciéndolo a binario mediante el ensamblador de CE4301 (Sec. 5.3)

como paso final de su propia cadena de herramientas; la estrategia interna queda a criterio del grupo de CE1108. Esto incluye la calendarización estática de instrucciones dentro de los bundles (asignación de operaciones a slots, manejo de dependencias de datos y de control, e inserción de NOPs donde corresponda), dado que, como se indicó en la Sec. 4.1, el hardware no resuelve estos riesgos de forma automática.

6.1. Contrato de Interfaz ISA–Compilador

Independientemente de la estrategia, herramientas o lenguaje de implementación que el grupo de CE1108 decida utilizar para su compilador, el grupo de CE4301 debe garantizar que sea posible **escribir y ejecutar programas directamente en el lenguaje ensamblador de la arquitectura propia**, de forma completamente independiente de dicho compilador. En particular:

- El ensamblador propio (Sec. 5.3) debe funcionar de forma autónoma, sin depender del compilador de CE1108, y debe permitir ejercitar el 100 % de las instrucciones del set diseñado mediante programas escritos a mano.
- El grupo de CE4301 debe entregar a su contraparte de CE1108 una **especificación completa y estable** del ISA y del formato de bundles (*instruction reference sheet*), incluyendo la **codificación binaria exacta** de cada instrucción y de cada slot (necesaria para que CE1108 pueda emitir binario directamente), así como la sintaxis de su ensamblador propio, para que sirva de referencia al momento de generar código.
- El grupo de CE4301 debe poner a disposición de su contraparte el modelo de simulación en SystemVerilog (o una interfaz reproducible sobre este) para que el código generado por el compilador pueda ejecutarse y validarse sobre el procesador real.
- Cualquier cambio al ISA o al formato de bundles que ocurra después de establecido este contrato debe comunicarse formalmente al grupo de contraparte, dado su impacto directo en el generador de código.

Esta separación permite validar la arquitectura de forma aislada (mediante el ensamblador propio), sin bloquear el avance de CE4301 ante retrasos en el compilador de la contraparte, a la vez que exige una coordinación real entre ambos equipos para lograr la integración completa.

6.2. Validación Conjunta

Antes de la defensa final, cada grupo de CE4301 deberá ejecutar sobre su procesador VLIW al menos un programa generado por el compilador de su contraparte de CE1108 (por ejemplo, una rutina que invoque el cifrado/descifrado Feistel4 desde código de alto nivel), demostrando bundles correctamente formados, respeto de las dependencias de datos/control planificadas por el compilador, y resultados correctos verificados contra la implementación de referencia.

7. Entregables

Los entregables se organizan según las dos entregas oficiales definidas en la Sec. 3.

1. **Especificación del ISA (Entrega 1, 25 de septiembre):** Documento detallando la arquitectura VLIW, formato de bundle y slots, set de instrucciones, convenciones de registros, etc. Puede entregarse en **formato Markdown** (*isa.md*, el mismo archivo requerido más abajo en la Documentación Técnica), siempre que esté enriquecido con tablas y diagramas/imágenes (formato de bundle, codificación de instrucciones, etc.) que hagan la especificación clara y completa; no es necesario un documento en LaTeX o PDF.

2. **Código SystemVerilog (Entrega 2, 14 de octubre):** Todos los módulos del procesador implementados en SystemVerilog, incluyendo:

   - Archivo principal: top.sv
   - Módulos: alu.sv, lsu.sv, bru.sv, crypto_unit.sv, key_vault.sv, dispatch.sv, regfile.sv, memory.sv, etc.
   - Testbenches: tb_top.sv, tb_alu.sv, tb_crypto_unit.sv, tb_key_vault.sv, etc.
   - Makefile para compilar con Icarus Verilog y/o Verilator y ejecutar simulaciones
   - Ensamblador propio y herramienta de carga de archivos, como requerimientos funcionales

3. **Documentación Técnica (Entrega 2, 14 de octubre):** En directorio *docs/* con archivos markdown (.md):

   - *isa.md*: Especificación detallada del ISA y del formato de bundles, instruction reference sheet.
   - *microarchitecture.md*: Diagrama de bloques y organización interna (pipeline, dispatch, slots, unidades funcionales).
   - *simulation.md*: Descripción del modelado de la simulación.
   - *compiler-integration.md*: Descripción del contrato de interfaz con el grupo de CE1108 y evidencia de la validación conjunta.

4. **README.md (Entrega 2, 14 de octubre):** Con instrucciones de compilación, ejecución, estructura del proyecto, ejemplos de uso del ensamblador propio y de la herramienta de carga (demostrando carga de diferentes tipos de archivos y operaciones de cifrado/-descifrado), y cualquier información necesaria para usar el código.

8. Evaluación del proyecto

La evaluación se organiza en dos bloques, correspondientes a las dos entregas oficiales (Sec. 3). El diseño del ISA se evalúa **una única vez**, como parte de la Entrega 1; la Entrega 2 no vuelve a calificar ese contenido, sino su consistencia con lo implementado.

8.1. Evaluación Entrega 1 – Especificación del ISA (20 %)

Se evalúa el documento entregado el 25 de septiembre (Sec. 3), mediante revisión del documento y una breve consulta al grupo (a discreción del profesor, no necesariamente una defensa presencial completa):

1. *Instruction reference sheet* o *green sheet* del ISA, incluyendo el formato de bundle/slots y la codificación binaria exacta de cada instrucción (necesaria para la Sec. 6).

2. Justificación de las decisiones de diseño: unidades funcionales y su asignación a slots, modos de direccionamiento, tipos de datos, estrategia frente a saltos, y diseño de las instrucciones de la unidad criptográfica y de la bóveda de llaves (Sec. 4.4).

8.2. Evaluación Entrega 2 – Implementación Funcional y Defensa Final (80 %)

La defensa se realizará de **forma presencial** en un horario a definir (preferiblemente en horas de clase), en un espacio de 25 minutos aproximadamente. Deberá mostrarse el sistema en funcionamiento: compilación sin errores, simulación exitosa (Icarus Verilog o Verilator), ejecución de todas las instrucciones (ALU, memoria, control de flujo, Feistel4, bóveda de llaves), validación con la herramienta de carga y cifrado de archivos, consistencia con la especificación de la Entrega 1, e integración con el compilador de la contraparte de CE1108.

8.2.1. Presentación proyecto funcional (50 %)

1. **Microarquitectura/Implementación (20 %)**:

   a) Diagrama de bloques de la microarquitectura (dispatch, slots, unidades funcionales, bóveda de llaves, banco de registros, memoria).

   b) Organización **segmentada (pipeline)** correctamente implementada, con bundles en distintas etapas simultáneamente (Sec. 4.1.3).

   c) Ejecución correcta y en paralelo de las unidades funcionales dentro de un mismo bundle.

   d) Estructura modular del código SystemVerilog.

2. **Validación de la Aplicación Criptográfica (20 %)**:

   a) Cifrado Feistel4 de datos cargados en memoria (archivo genérico y, opcionalmente, imagen).

   b) Descifrado y verificación de integridad contra la implementación de referencia.

   c) Manejo correcto de la bóveda de llaves: almacenamiento de las 4 llaves, control de acceso (autenticación/autorización) y cumplimiento de las restricciones de aislamiento (Sec. 4.4.1).

   d) Ejecución de programas VLIW que combinan instrucciones tradicionales y criptográficas en los mismos bundles.

3. **Integración con Compiladores – CE1108 (10 %)**[1]:

   a) Ensamblador propio funcional, capaz de ejercitar el 100 % del set de instrucciones diseñado, de forma independiente del compilador de la contraparte.

[1] Este 10 % corresponde exactamente a un 20 % dentro del rubro *Presentación proyecto funcional* (50 %) de la Entrega 2; la evaluación del componente de CE1108 es responsabilidad del profesor de dicho curso.

b) Especificación de la ISA (incluyendo codificación binaria) entregada formalmente al grupo de contraparte de CE1108 (contrato de interfaz).

c) Ejecución correcta, sobre el procesador VLIW propio, de al menos un programa generado por el compilador de la contraparte de CE1108.

8.2.2. Repositorio y documentación de diseño (30 %)

1. **Gestión de repositorio (15 %)**:

   a) **Uso adecuado de Git con flujo de trabajo profesional (8 %)**: El proyecto debe mantener un repositorio Git con historial de commits descriptivos, uso de ramas de desarrollo (dev branches) para nuevas funcionalidades. Se evaluará la calidad de los mensajes de commit, la organización de branches, y el uso de pull requests o merge requests.

   b) **README completo y claro (4 %)**: El repositorio debe incluir un archivo README detallado que explique el propósito del proyecto, instrucciones de compilación y ejecución (Icarus Verilog y/o Verilator), dependencias necesarias, estructura del código, y cualquier otra información relevante para entender y utilizar el código, así como ejemplos de uso con archivos de prueba.

   c) **Gestión de issues y milestones (3 %)**: Demostrar el uso de issues para tracking de funcionalidades, bugs y mejoras, organizados en milestones que reflejen el progreso del proyecto.

2. **Documentación de diseño (15 %)**: La documentación del diseño deberá ser agregada al repositorio, en un directorio *docs*, en formato markdown (.md). El archivo *isa.md* debe mantenerse consistente con lo evaluado en la Entrega 1, pero **no se recalifica su contenido aquí**; se evalúan los siguientes documentos:

   a) **Organización/Microarquitectura (5 %)**: Descripción de la organización interna del procesador en SystemVerilog, incluyendo diagrama de bloques que muestre los componentes principales (fetch, dispatch/decode multi-slot, ALU, LSU, BRU, unidad criptográfica, bóveda de llaves, banco de registros, memoria) y sus interconexiones. Explicación del flujo de datos e instrucciones dentro del procesador y su implementación en módulos de SystemVerilog.

   b) **Modelado del software/Simulación (5 %)**: Descripción del modelo de simulación implementado, incluyendo detalles sobre la implementación del banco de registros, unidades funcionales, memoria, ensamblador propio, y la interacción con la herramienta de carga de archivos.

   c) **Integración con Compiladores – CE1108 (5 %)**: Descripción del contrato de interfaz con el grupo de contraparte (Sec. 6) y evidencia de la validación conjunta (*compiler-integration.md*).

9. Notas importantes

- El proyecto debe compilar y simular sin errores con **Icarus Verilog o Verilator**.

- La simulación debe ejecutarse exitosamente, demostrando todas las funcionalidades, tanto "tradicionales" (ALU, memoria, control de flujo) como criptográficas (Feistel4 y bóveda de llaves).
- Todo el código debe incluir comentarios descriptivos en secciones críticas.
- Se recomienda usar un Makefile para facilitar compilación y ejecución de testbenches.
- La coordinación con el grupo de contraparte de CE1108 debe iniciarse tan pronto como el ISA alcance una versión estable, para no comprometer los tiempos de desarrollo del compilador.

Referencias

[1] Horst Feistel. Cryptography and computer privacy. *Scientific American*, 228(5):15–23, 1973.